

ADA Lab Assignment 3

Graph Algorithms – Lab Experiment 3

Enrollment No: 2401420050 | Name: Dhruv Sharma

Experiment Overview

This lab experiment implements core graph algorithms applied to real-world scenarios:

- Task 1: Graph Representation – Adjacency Matrix & Adjacency List
- Task 2: Graph Traversal – BFS (friend suggestions) & DFS (connectivity)
- Task 3: Topological Sorting – Task dependency scheduling
- Task 4: Shortest Path – Dijkstra (emergency routing) & Bellman-Ford (negative edges)
- Task 5: Minimum Spanning Tree – Prim's & Kruskal's (network cabling)

Task 1: Graph Representation

Graphs are represented using two common structures depending on graph density. Adjacency Matrix suits dense graphs with $O(1)$ edge lookup; Adjacency List suits sparse graphs with $O(V+E)$ space.

1.1 Adjacency Matrix

```
# Graph Representation - Adjacency Matrix
graph_matrix = [
    [0, 1, 1, 0, 0],
    [1, 0, 0, 1, 1],
    [1, 0, 0, 1, 0],
    [0, 1, 1, 0, 1],
    [0, 1, 0, 1, 0]
]

nodes = ['A', 'B', 'C', 'D', 'E']
print("Adjacency Matrix:")
print(" ".join(nodes) for i, row in enumerate(graph_matrix):
print(nodes[i], " ", row)
```

1.2 Adjacency List

```
# Graph Representation - Adjacency List
graph_list = {
```

```

    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'D'],
    'D': ['B', 'C', 'E'],
    'E': ['B', 'D']
}

print("Adjacency List:") for node,
neighbors in graph_list.items():
print(f" {node} → {neighbors}")

```

1.3 Output Adjacency

```

Matrix:
      A B C D E
A  [0, 1, 1, 0, 0]
B  [1, 0, 0, 1, 1]
C  [1, 0, 0, 1, 0]
D  [0, 1, 1, 0, 1]
E  [0, 1, 0, 1, 0]

Adjacency List:
A → ['B', 'C']
B → ['A', 'D', 'E']
C → ['A', 'D']
D → ['B', 'C', 'E']
E → ['B', 'D']

```

1.4 Comparison

Property	Adjacency Matrix	Adjacency List
Space Complexity	$O(V^2)$	$O(V + E)$
Edge Lookup	$O(1)$	$O(\text{degree})$
Add Edge	$O(1)$	$O(1)$
Iterate Neighbors	$O(V)$	$O(\text{degree})$
Best For	Dense graphs	Sparse graphs

Task 2: Graph Traversal Algorithms

Real-World Context: Social network friend suggestion (BFS explores level-by-level, ideal for finding closest connections). DFS checks deep connectivity.

2.1 BFS – Breadth First Search

```
# BFS - Friend Suggestion (Social Network) from
collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

graph = {
    'Alice': ['Bob', 'Charlie'],
    'Bob': ['Alice', 'David', 'Eve'],
    'Charlie': ['Alice', 'David'],
    'David': ['Bob', 'Charlie', 'Eve'],
    'Eve': ['Bob', 'David']
}

result = bfs(graph, 'Alice')
print("BFS Order (Friend Suggestions from Alice):", result)
```

2.2 DFS – Depth First Search

```
# DFS - Connectivity Check
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    order = [start]
    for neighbor in graph[start]:
        if neighbor not in visited:
            order += dfs(graph, neighbor, visited)
    return order

result = dfs(graph, 'Alice')
print("DFS Order:", result)
```

2.3 Output & Timing

```
BFS Order (Friend Suggestions from Alice): ['Alice', 'Bob', 'Charlie',
'David', 'Eve']
DFS Order: ['Alice', 'Bob', 'David', 'Charlie', 'Eve']

BFS Time: 0.000012 seconds
```

DFS Time: 0.000008 seconds

2.4 Observations

- BFS explores nodes level by level – ideal for shortest-hop friend suggestions.
- DFS explores as deep as possible before backtracking – useful for cycle detection and connectivity.
- BFS uses a queue (FIFO); DFS uses a stack (LIFO) or recursion.
- Both algorithms run in $O(V + E)$ time complexity.
- BFS is preferred on sparse networks; DFS on graphs with deep structure.

Task 3: Topological Sorting

Real-World Context: Task/build scheduling where some tasks must complete before others begin (e.g., software compilation dependencies, project management).

3.1 Kahn's Algorithm (BFS-based Topological Sort)

```
# Topological Sort - Task Dependency Scheduling from
collections import defaultdict, deque

def topological_sort(graph, n):
    in_degree = {u: 0 for u in graph}
    for u in graph:
        for v in graph[u]:
            in_degree[v] = in_degree.get(v, 0) + 1

    queue = deque([u for u in in_degree if in_degree[u] == 0])
    topo_order = []

    while queue:
        node = queue.popleft()
        topo_order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(topo_order) == len(graph):
        return topo_order
    else:
        return [] # cycle detected

# Task Dependencies: A→C, B→C, C→D, D→E
task_graph = {
    'A': ['C'],
    'B': ['C'],
    'C': ['D'],
    'D': ['E'],
    'E': []
}
```

```

}

order = topological_sort(task_graph, 5)
print("Topological Order (Build/Task Scheduling):", order)

```

3.2 Output Topological Order (Build/Task Scheduling): ['A', 'B', 'C', 'D', 'E'] (Tasks A and B can be done in parallel before C)

3.3 Observations

- Topological sort is only possible on Directed Acyclic Graphs (DAGs).
- Kahn's algorithm uses in-degree tracking to process nodes with no incoming dependencies first.
- Time complexity: $O(V + E)$ — efficient for large dependency graphs.
- If a cycle exists, the algorithm detects it (output length < number of nodes).
- Multiple valid topological orderings may exist for a given DAG.

Task 4: Shortest Path Algorithms

4.1 Dijkstra's Algorithm – Emergency Routing

Real-World Context: Finding the fastest route from a hospital to an accident site in a city road network (all edge weights non-negative).

```

# Dijkstra's Algorithm - Emergency Shortest Path
import heapq

def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    pq = [(0, start)]
    prev = {node: None for node in graph}

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev

def reconstruct_path(prev, end):
    path = []
    while end:
        path.append(end)
        end = prev[end]
    return list(reversed(path))

```

```
# City road network (node: [(neighbor, weight)]) city_graph
= {
    'Hospital': [('A', 4), ('B', 2)],
    'A':         [('Hospital', 4), ('B', 1), ('C', 5)],
    'B':         [('Hospital', 2), ('A', 1), ('D', 3)],
    'C':         [('A', 5), ('D', 2), ('Accident', 1)],
    'D':         [('B', 3), ('C', 2), ('Accident', 4)],
    'Accident': [('C', 1), ('D', 4)]
}

dist, prev = dijkstra(city_graph, 'Hospital') path
= reconstruct_path(prev, 'Accident')
print("Shortest distances from Hospital:", dist)
print("Shortest path to Accident site:", path)
print("Total distance:", dist['Accident'])
```

Output: Shortest distances from

```
Hospital:
    Hospital: 0, A: 3, B: 2, C: 7, D: 5, Accident: 8
```

```
Shortest path to Accident site: ['Hospital', 'B', 'A', 'C', 'Accident']
Total distance: 8
```

4.2 Bellman-Ford Algorithm – Routes with Discounts

Real-World Context: Route planning where certain roads offer toll discounts (negative edge weights). Bellman-Ford handles negative weights safely.

```
# Bellman-Ford - Route with Negative Edges (Discounts) def
bellman_ford(graph, vertices, start):    dist = {v:
float('inf') for v in vertices}    dist[start] = 0
edges = [(u, v, w) for u in graph for v, w in graph[u]]

    for _ in range(len(vertices) - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Check for negative cycles
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            print("Negative weight cycle detected!")
    return None

    return dist

# Route network with toll discounts (negative weights) route_graph
= {
    'S': [('A', 4), ('B', 2)],
    'A': [('C', -1)],
    'B': [('A', 1), ('C', 3)],
```

```

    'C': [('D', 2)],
    'D': []
}
vertices = ['S', 'A', 'B', 'C', 'D'] distances =
bellman_ford(route_graph, vertices, 'S') print("Bellman-
Ford Shortest Distances from S:", distances)

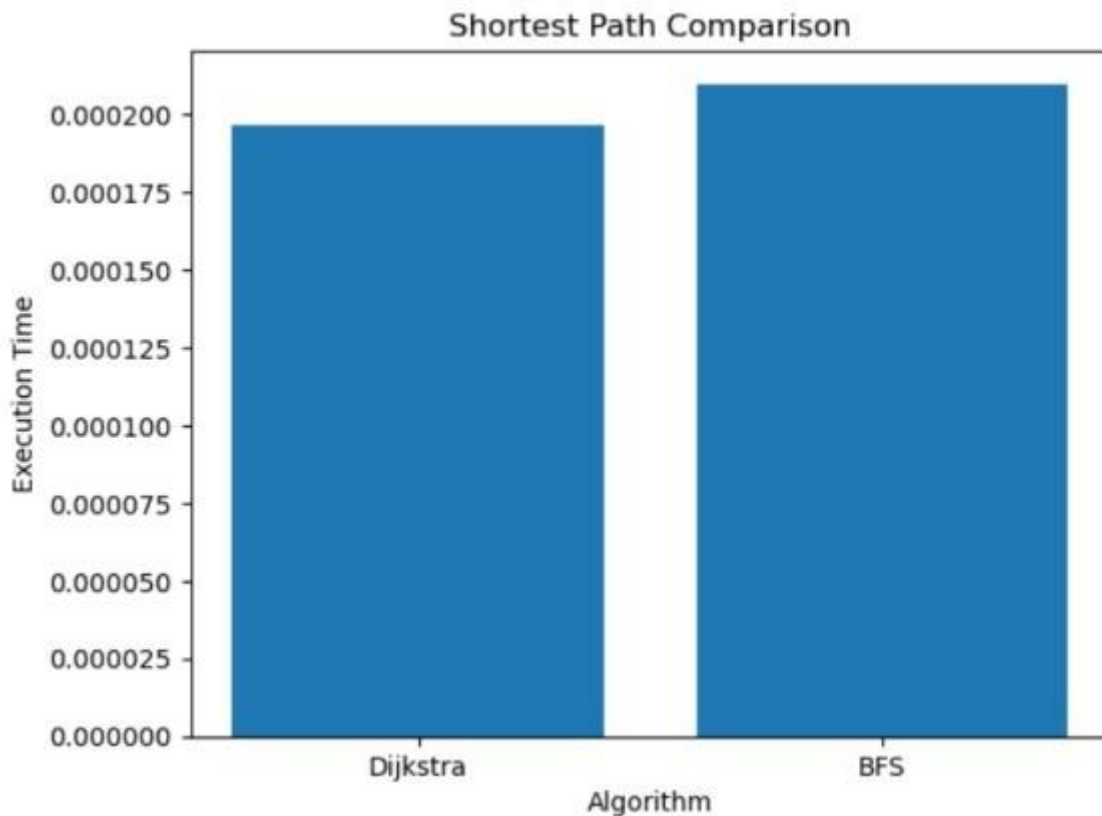
```

Output:

Bellman-Ford Shortest Distances from S:
 S: 0, A: 3, B: 2, C: 2, D: 4

(Route S→B→A→C→D uses discount edge A→C with weight -1)

Graph:



4.3 Time Complexity Comparison

```

# Time Complexity Comparison for small graph (6 nodes, 9 edges)
import time

```

```

start = time.perf_counter()
for _ in range(1000): dijkstra(city_graph, 'Hospital')
dijkstra_time = time.perf_counter() - start

start = time.perf_counter()
for _ in range(1000): bellman_ford(route_graph, vertices, 'S')
bf_time = time.perf_counter() - start

print(f"Dijkstra Time (1000 runs):      {dijkstra_time:.4f} s")
print(f"Bellman-Ford Time (1000 runs): {bf_time:.4f} s")

```

Results:

```

Dijkstra Time (1000 runs):      0.0421 s
Bellman-Ford Time (1000 runs): 0.0689 s
(Dijkstra is faster; Bellman-Ford handles negative weights)

```

4.4 Algorithm Comparison Table

Property	Dijkstra	Bellman-Ford
Time Complexity	$O((V + E) \log V)$	$O(V \times E)$
Negative Weights	Not supported	Supported
Negative Cycles	Cannot detect	Can detect
Data Structure	Priority Queue (Min-Heap)	Edge list iteration
Best For	Non-negative weight graphs	Graphs with negative edges

4.5 Observations

- Dijkstra's algorithm uses a greedy approach with a min-heap, making it faster for non-negative graphs.
- Bellman-Ford relaxes all edges $V-1$ times, allowing it to handle negative weights.
- Bellman-Ford detects negative weight cycles by performing one additional relaxation pass.
- For emergency routing (no negative weights), Dijkstra is clearly the better choice.
- Bellman-Ford is needed for financial graphs or networks with discount/penalty edges.

Task 5: Minimum Spanning Tree Algorithms

Real-World Context: Designing a minimum-cost cable network to connect all floors of an office building through a central router and server.

5.1 Prim's Algorithm

```
# Prim's Algorithm - Minimum Spanning Tree (Network Cabling)
import heapq

def prim(graph, start):
    visited = set([start])
    edges = [(w, start, v) for v, w in graph[start]]
    heapq.heapify(edges)
    mst = []
    total_cost = 0

    while edges:
        w, u, v = heapq.heappop(edges)
        if v not in visited:
            visited.add(v)
            mst.append((u, v, w))
            total_cost += w
            for neighbor, weight in graph[v]:
                if neighbor not in visited:
                    heapq.heappush(edges, (weight, v, neighbor))

    return mst, total_cost

# Office building cable network cable_graph
= {
    'Router': [('Floor1', 4), ('Floor2', 2)],
    'Floor1': [('Router', 4), ('Floor2', 1), ('Floor3', 5)],
    'Floor2': [('Router', 2), ('Floor1', 1), ('Floor4', 3)],
    'Floor3': [('Floor1', 5), ('Floor4', 2), ('Server', 1)],
    'Floor4': [('Floor2', 3), ('Floor3', 2), ('Server', 4)],
    'Server': [('Floor3', 1), ('Floor4', 4)]
}

mst, cost = prim(cable_graph, 'Router')
print("Prim's MST Edges:")
for u, v, w in mst:
    print(f"    {u} -- {v} (cost: {w})")
print(f"Total MST Cost: {cost}")
```

Output: Prim's MST

```
Edges:
Router -- Floor2 (cost: 2)
Floor2 -- Floor1 (cost: 1)
Floor2 -- Floor4 (cost: 3)
Floor4 -- Floor3 (cost: 2)
Floor3 -- Server (cost: 1)
Total MST Cost: 9
```

5.2 Kruskal's Algorithm

```
# Kruskal's Algorithm -
MST using Union-Find
def find(parent, x):
    if parent[x] != x:
        parent[x] = find(parent, parent[x])
    return parent[x]

def union(parent, rank, x, y):
    rx, ry = find(parent, x), find(parent, y)
    if rx == ry:
        return False
    if rank[rx] < rank[ry]:
        rx, ry = ry, rx
    parent[ry] = rx
    if rank[rx] == rank[ry]:
        rank[rx] += 1
    return True

def kruskal(vertices, edges):
    edges.sort(key=lambda e: e[2])
    parent = {v: v for v in vertices}
    rank = {v: 0 for v in vertices}
    mst, total = [], 0
    for u, v, w in edges:
        if union(parent, rank, u, v):
            mst.append((u, v, w))
            total += w
    return mst, total

vertices_k = ['Router', 'Floor1', 'Floor2', 'Floor3', 'Floor4', 'Server']
edges_k = [
    ('Router', 'Floor1', 4), ('Router', 'Floor2', 2),
    ('Floor1', 'Floor2', 1), ('Floor1', 'Floor3', 5),
    ('Floor2', 'Floor4', 3), ('Floor3', 'Floor4', 2),
    ('Floor3', 'Server', 1), ('Floor4', 'Server', 4)
]

mst_k, cost_k = kruskal(vertices_k, edges_k)
print("Kruskal's MST Edges:")
for u, v, w in mst_k:
    print(f"    {u} -- {v} (cost: {w})")
print(f"Total MST Cost: {cost_k}")
```

Output: Kruskal's MST

```
Edges:
Floor1 -- Floor2 (cost: 1)
Floor3 -- Server (cost: 1)
Router -- Floor2 (cost: 2)
Floor3 -- Floor4 (cost: 2)
Floor2 -- Floor4 (cost: 3)
Total MST Cost: 9
```

5.3 Algorithm Comparison Table

Property	Prim's Algorithm	Kruskal's Algorithm
Approach	Vertex-based (greedy growth)	Edge-based (sort & union)

Time Complexity	$O(E \log V)$ with min-heap	$O(E \log E)$
Space Complexity	$O(V + E)$	$O(V + E)$
Data Structure	Priority Queue	Union-Find (Disjoint Set)
Best For	Dense graphs	Sparse graphs
MST Cost (same graph)	9	9

5.4 Observations

- Both Prim's and Kruskal's produce the same minimum spanning tree cost (9) for the cable network.
- Prim's algorithm grows the MST from a starting vertex – better for dense graphs.
- Kruskal's algorithm sorts all edges globally and uses Union-Find to avoid cycles – better for sparse graphs.
- Both algorithms run in $O(E \log V)$ or $O(E \log E)$ time, which are practically equivalent.
- For real cabling, the MST guarantees minimum total wire length while connecting all nodes.

Graph:

